
Table of Contents

hw2_me501_Q2c.m	1
Setup: constants and initial conditions	1
Simulate to obtain trajectory - Numerical, approximated!	1
Exact Solution for comparison	2
Simulate as a discrete time linear system	2
Plot	2

hw2_me501_Q2c.m

(C) Andrew Sabelhaus, 2026

```
clear all;
close all;
clc;

disp('ME 501 HW2 simulation of pneumatic chambers dynamics, comparison with
exact solution and discretized dynamical system.')

ME 501 HW2 simulation of pneumatic chambers dynamics, comparison with exact
solution and discretized dynamical system.
```

Setup: constants and initial conditions

```
tmax = 5;
dt = 0.05;

n = tmax/dt; % number of timesteps, for loop-ing
% somehow matlab doesn't round correctly. sometimes...
n = round(n);

% The matrix for our linear dynamics is
A = [-3  1  0;
     1 -2  1;
     0  1 -3];

%%%% Initial conditions:

p1_0 = 5;
p2_0 = 0;
p3_0 = 50;
x0 = [p1_0; p2_0; p3_0];
```

Simulate to obtain trajectory - Numerical, approximated!

```
% insert our initial condition as the 1st element in traj
% note that traj will have n+1 columns.
```

```

x_traj = zeros(size(x0,1), n+1);
x_traj(:,1) = x0;

% actual simulation! Use Forward Euler for now.
for t=1:n
    %%%% DYNAMICS
    bolddotx_t = A*x_traj(:,t);
    % Forward Euler
    x_traj(:,t+1) = x_traj(:,t) + dt*bolddotx_t;
end

```

Exact Solution for comparison

```

% let's precalculate for a one-to-one comparison
x_traj_exact = zeros(size(x0,1), n+1);
x_traj_exact(:,1) = x0;

% precalculate via iteration
for k=1:n
    % The exact solution is  $x(t) = e^{(A)t}x_0$ 
    % Since "t" is inside the exponential, need to re-call this function
    % each iteration.
    t = k*dt; % here, k is index, dt is timestep, so total time is t.
    %%%% If exponentiating at each timestep:
    Phi_t = expm(A*t);
    x_traj_exact(:,k+1) = Phi_t*x0;
end

```

Simulate as a discrete time linear system

```

% Our claim is that  $x(k) = A_d^k x(0)$ , where  $A_d = e^{(A*\text{deltaT})}$ 
A_d = expm(A*dt);

% let's precalculate for a one-to-one comparison
x_traj_d = zeros(size(x0,1), n+1);
x_traj_d(:,1) = x0;

% the iteration is even simpler now!
for k=1:n
    x_traj_d(:,k+1) = A_d^k * x0;
    % For your own experiment: try this one out, you should get the same
    % result! Can you reason out why that will happen, using linear
    % algebra?
    % x_traj_d(:,k+1) = A_d * x_traj_d(:,k);
end

```

Plot

```

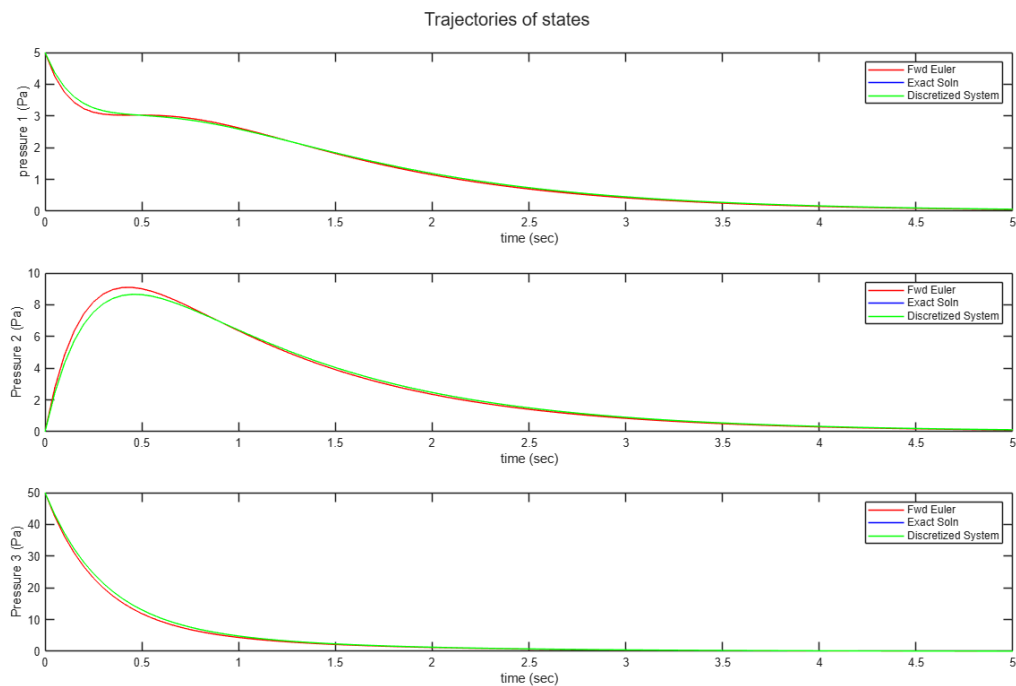
% We can also plot the three time series trajectories of theta, dottheta,
% and u
figure;
hold on;

```

```

t = [0:dt:tmax];
subplot(3,1,1);
sgtitle('Trajectories of states');
% pressure 1
plot(t, x_traj(1,:), 'r');
hold on;
plot(t, x_traj_exact(1,:), 'b')
plot(t, x_traj_d(1,:), 'g')
legend(["Fwd Euler", "Exact Soln", "Discretized System"]);
xlabel('time (sec)');
ylabel('pressure 1 (Pa)');
% pressure 2
subplot(3,1,2);
plot(t, x_traj(2,:), 'r');
hold on;
plot(t, x_traj_exact(2,:), 'b')
plot(t, x_traj_d(2,:), 'g')
legend(["Fwd Euler", "Exact Soln", "Discretized System"]);
xlabel('time (sec)');
ylabel('Pressure 2 (Pa)');
% pressure 3
subplot(3,1,3);
plot(t, x_traj(3,:), 'r');
hold on;
plot(t, x_traj_exact(3,:), 'b')
plot(t, x_traj_d(3,:), 'g')
legend(["Fwd Euler", "Exact Soln", "Discretized System"]);
xlabel('time (sec)');
ylabel('Pressure 3 (Pa)');

```



Published with MATLAB® R2026a